



ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba



UNIVERSIDAD DE CÓRDOBA

Plataformas Big Data

Curso 2026/2027

Boletín de Prácticas

Prof: Dr. Héctor Martínez Pérez

Dpto. Ingeniería Electrónica y Computadores

Arquitectura de Computadores, Electrónica y Tecnología Electrónica

2º Cuatrimestre

Código: 283007

PRÁCTICA 1

Almacenamiento de los datos en memoria

CONTEXTO

En la actualidad, el crecimiento de la inteligencia artificial (IA), el análisis masivo de datos y los servicios en la nube han incrementado de forma exponencial la necesidad de procesar y almacenar grandes volúmenes de información. Empresas como Google, Amazon, Microsoft o Meta disponen de centros de procesamiento de datos (CPD) formados por miles de servidores que ejecutan aplicaciones y almacenan la información de millones de usuarios de forma continua.

Estos CPD no solo prestan servicios como motores de búsqueda, comercio electrónico, asistentes inteligentes o plataformas de almacenamiento en la nube, sino que también se utilizan para ejecutar aplicaciones científicas y de ingeniería de gran complejidad. Entre ellas destacan el entrenamiento de modelos de inteligencia artificial, el procesamiento de grandes bases de datos, la resolución de problemas de álgebra lineal, las simulaciones numéricas, los estudios genómicos o la predicción meteorológica, entre muchas otras.

En todas estas aplicaciones, el rendimiento computacional es un factor determinante. Sin embargo, antes de optimizar un algoritmo o distribuir el trabajo entre múltiples servidores, es imprescindible comprender cómo se almacenan los datos en memoria. Una organización adecuada de la información permite reducir el consumo de memoria, mejorar la localidad espacial y temporal de los datos, disminuir el número de fallos de caché y aumentar el rendimiento del procesador durante su acceso.

OBJETIVOS

Los principales objetivos de esta práctica son:

- Conocer la importancia de ciertas operaciones de álgebra lineal en el panorama actual de la Inteligencia Artificial y plataformas Big Data para así entender la importancia del acceso a los datos en memoria.
- Comprender cómo se almacena de forma eficiente los datos en memoria.
- Analizar la organización contigua de datos en memoria.
- Entender el concepto de localidad espacial.

EJERCICIO 1

En primer lugar, es de vital importancia comprender la base de la IA, y qué operaciones fundamentales se llevan a cabo en los modelos de redes neuronales más usados en la actualidad.

Existen multitud de modelos de redes neuronales:

- **Redes neuronales convolucionales:** utilizadas por la comunidad para la predicción de eventos, estadísticas, detección de objetos, segmentación, etc.
- **Transformers encoders y decoders:** los LLM tuvieron un gran impacto en la sociedad al poder llevar a cabo conversaciones con los usuarios de todo el mundo y poder llevar a cabo tareas complejas a nivel de desarrollo de software, diseño gráfico, traducciones de texto, etc.
- **Redes neuronales basadas en grafos:** otro tipo de redes neuronales basadas en grafos destinadas al estudio de estadísticas y relaciones de datos (citas en artículos científicos, estadísticas de compras en Amazon, relaciones de amistades en Facebook y otras redes sociales, etc.).

Existe un número mucho mayor de modelos de redes neuronales, además, de multitud de variantes que cada año se van multiplicando y haciendo estos modelos más eficientes y también más complejos. Cada modelo de red neuronal contiene sus propias operaciones, pero todos ellos parten de una característica común:

**LA OPERACIÓN MÁS COSTOSA DE FORMA GLOBAL ES EL PRODUCTO MATRICIAL
GEMM**

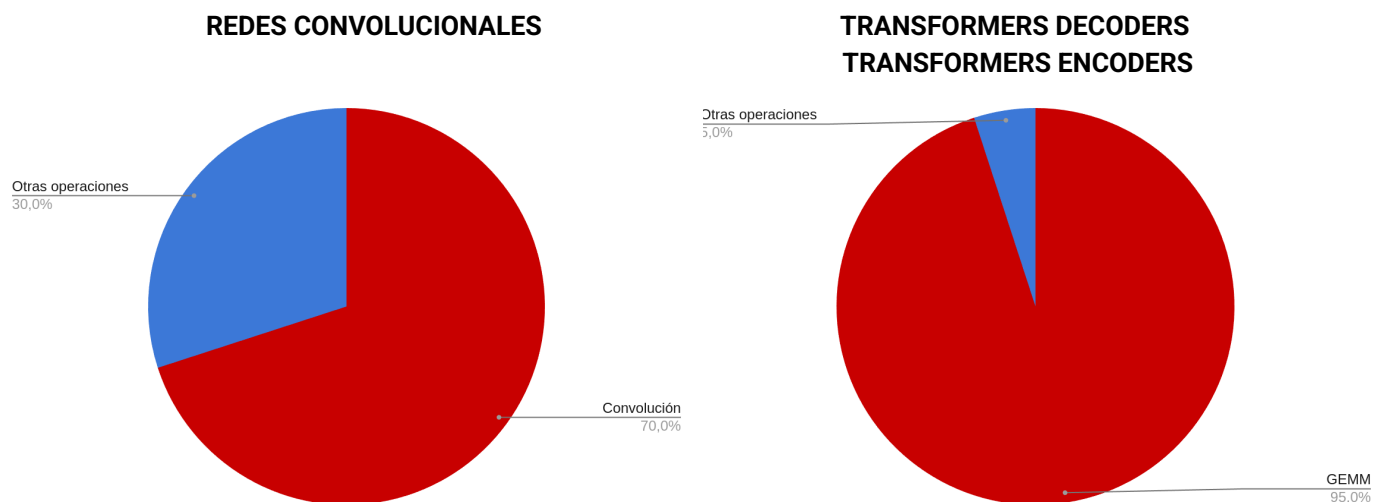


Figura 1. Coste del producto matricial y convolucional en diferentes tipos de modelos de IA.

La imagen de la izquierda muestra que alrededor del **65%** del tiempo de cómputo al realizar una red convolucional se consume en la operación de convolución. Por motivos de paralelismo y eficiencia de rendimiento, es común que la mayoría de las bibliotecas de álgebra lineal destinadas al procesamiento de este tipo de operaciones (BLIS, OpenBLAS, MKL, XNNPACK, ARMCL, ARMPL, etc.) usen el método ***im2col/im2row*** para convertir la operación de convolución en una **GEMM**.

La imagen de la derecha muestra cómo, en el proceso de inferencia de Transformers encoders o Transformers decoders (proceso que se lleva a cabo millones y millones de veces por segundo en la actualidad, ya que cada consulta a plataformas como ChatGPT, Claude, Gemini, etc. se realiza mediante este tipo de inferencia), la operación más costosa con diferencia es el producto matricial.

Dicho esto, nuestro objetivo va a ser trabajar en acelerar este tipo de operaciones de álgebra lineal al máximo.

Para comenzar, es de vital importancia disponer de los datos almacenados de forma adecuada. Para el primer ejercicio, el alumno debe conocer los dos principales tipos de almacenamiento de matrices en memoria: ***row-major*** y ***column-major***.

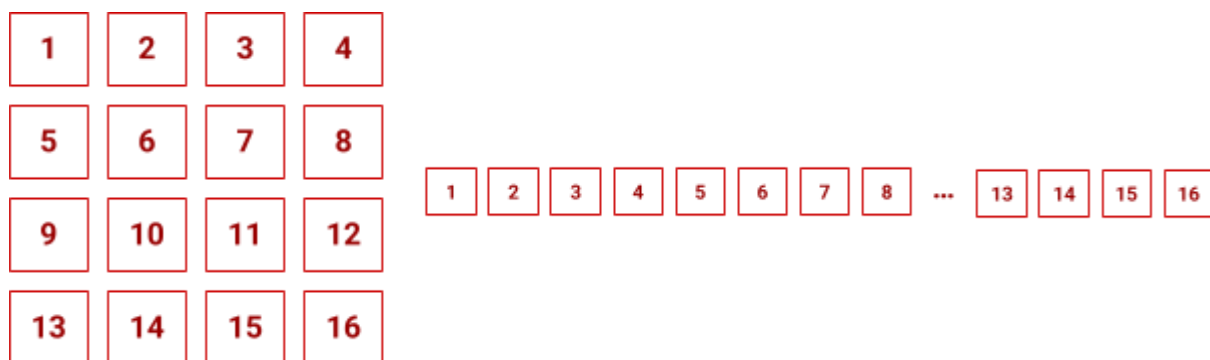


Figura 2. Almacenamiento de matriz basado en la estrategia *row-major*.

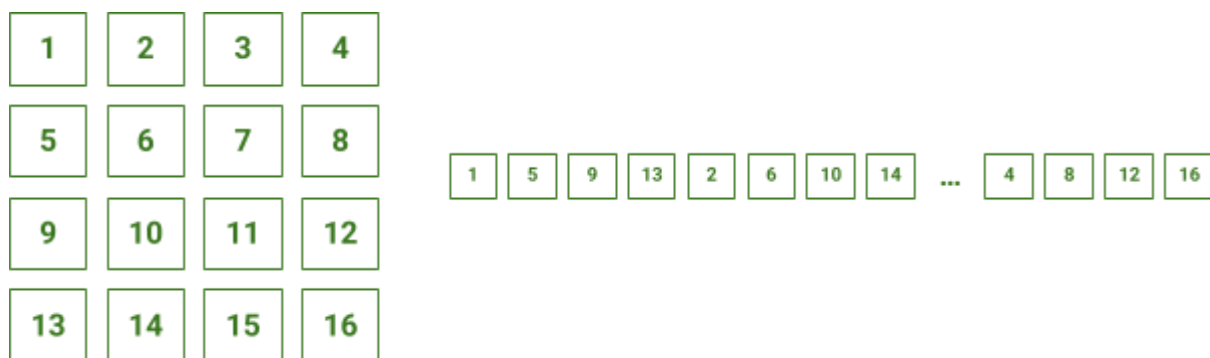


Figura 3. Almacenamiento de matriz basado en la estrategia *column-major*.

Con base en esto, el alumno deberá desarrollar en lenguaje de programación C/C++ un programa que cree 1 matriz de enteros M de tamaño $M=N=K=4$ (inicializada con valores aleatorios entre 0 y 10). La reserva de la matriz se llevará a cabo siguiendo el almacenamiento estándar conocido por el alumnado (doble puntero `[][]` o `**`). Además, se considerará que esta matriz original está almacenada por filas, es decir, siguiendo la estrategia *row-major*.

EJERCICIO 2

A continuación, se deberán implementar 2 rutinas:

```
void *row_major_transform(DTYPE **orig, DTYPE *dest,  
                        const size_t M, const size_t N);
```

```
void *row_major_transform(DTYPE **orig, DTYPE *dest, const size_t M,  
                        const size_t N);
```

Cada una de ellas recibirá un doble puntero a la matriz original y deberá transformar este formato de almacenamiento en memoria al formato que corresponda y en el buffer de destino pasado como parámetro. Para ello, recorrerá la matriz original por filas (almacenamiento que usamos por defecto) y se almacenará en el nuevo formato correspondiente y de manera serializada.

Para ellos, el alumno inicialmente llevará a cabo 2 reservas de memoria siguiendo el estándar de C (malloc) y con un único puntero (*). Seguidamente, se implementarán las rutinas y se llamarán pasando como parámetros los buffers creados.

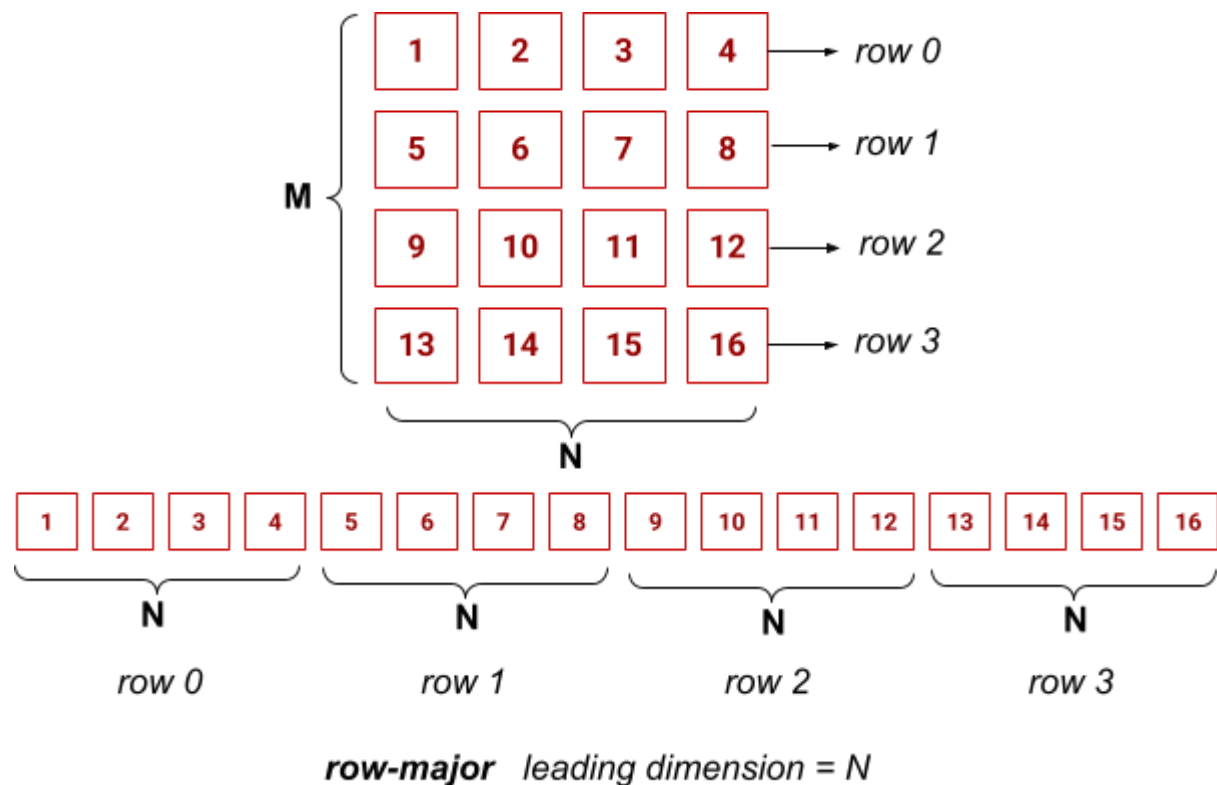


Figura 4. Esquema de almacenamiento de una matriz $M \times N$ en formato *row-major*.

EJERCICIO 3

Para comprobar el orden en el que se han almacenado los datos, en primer lugar, el alumno deberá mostrar por pantalla los valores que contienen cada una de las posiciones de los dos buffers rellenos por las rutinas anteriormente implementadas. Por pantalla deberá aparecer una secuencia de valores similares a las que aparecen en la parte derecha de la **Figura 2** y la **Figura 3**.

EJERCICIO 4

Seguidamente, el alumno mostrará por pantalla la matriz original, con formato similar al que aparece a la parte izquierda en las **Figuras 2 y 3**, así como las matrices almacenadas en cada uno de los buffers. En este caso, nótese que la matriz es la misma en los 3 escenarios y la diferencia es el tipo de almacenamiento, por este motivo, la salida por pantalla debe ser similar en los tres escenarios.

EJERCICIO 5

El alumno deberá mostrar por pantalla las direcciones de almacenamiento de cada uno de los elementos almacenados en las tres matrices generadas y reservadas en memoria y analizar el direccionamiento.

PRÁCTICA 2

Acceso de los datos en memoria

CONTEXTO

Una vez aprendidos los principales formatos de almacenamiento de matrices en memoria, es hora de abordar un caso real de uso. Como se ha comentado en la práctica anterior, la IA se aplica actualmente en multitud de campos: desarrollo de software, agricultura, física, química, biología, empresas privadas de diversos sectores, etc. Además, junto con la Computación de Altas Prestaciones (HPC) y el procesamiento del Big Data ha permitido entender y obtener información de millones y millones de datos que hasta hace poco tiempo carecían de sentido para la humanidad (procesamiento del Big Data).

Por todos estos motivos, es imprescindible conocer, tanto a alto como bajo nivel, el funcionamiento de las redes neuronales.

OBJETIVOS

En esta práctica el alumnado deberá implementar una red neuronal sencilla, desde cero, en lenguaje de programación C. Esto ayudará al alumnado a comprender las operaciones de álgebra lineal básicas que se llevan a cabo en el interior de una red neuronal, así como la complejidad computacional de las mismas.

Dicho esto, los objetivos principales de la práctica son:

- Comprensión del alumnado del funcionamiento de una red neuronal simple.
- Llevar a cabo de manera adecuada el acceso a los datos de entrada de una red neuronal.
- Conocimiento y comprensión de las funciones básicas que conforman una red neuronal simple.
- Manejar con soltura el acceso a los datos de una matriz almacenada en memoria serializada, tanto en formato *row-major* como en *column-major*.

EJERCICIO 1

La red neuronal más sencilla para implementar desde cero en C es un **Perceptrón Multicapa (MLP)** de una sola capa oculta. Las redes neuronales MLP se caracterizan por la siguiente arquitectura:

- **Capa de entrada:** Corresponde directamente al número de características o variables del problema.
- **Capa oculta:** Una única capa intermedia (3 - 5 neuronas) para aprender relaciones no lineales.
- **Capa de salida:** El resultado final de la red.

El funcionamiento de cualquier red neuronal es como el montaje en cadena de un producto. La red neuronal recibe como entrada los datos para ir pasando por diferentes etapas hasta generar una salida.

Nuestro MLP va a estar formado por dos funciones principales:

1. Suma ponderada (paso lineal). Esta rutina llevará a cabo el producto escalar entre las entradas y los pesos más el sesgo.

$$z = \left(\sum_{i=1}^n w_i \cdot x_i \right) + b$$

2. Función de activación (ReLU). Permite que la red aprenda relaciones complejas “rompiendo” la linealidad (convertirá valores negativos en cero).

$$f(z) = \max(0, z)$$

- **xi:** Son las entrada que vienen de la capa anterior.
- **wi:** Son los pesos almacenados provenientes del entrenamiento.
- **b:** Es el bias, el umbral de activación de la neurona.
- **z:** Es el resultado de la inferencia antes de ser filtrado por la activación.

Este primer ejercicio consistirá en que el alumno lleve a cabo las reservas correspondientes de memoria. Para ello, necesitaremos definir en 3 variables los tamaños principales de nuestras entradas y salidas:

- **BATCH**: Es el número de muestras por IN_DIM.
- **IN_DIM**: Es el número de valores por muestra que le entra a nuestra red neuronal.
 - En caso de una red de sensores, el valor de cada sensor.
 - En el caso de una imagen, los píxeles de una imagen.
 - etc.
- **OUT_DIM**: Número de neuronas de la capa actual

Para empezar, fijaremos:

- **BATCH=5**
- **IN_DIM=4096**
- **OUT_DIM=2048**

Seguidamente, el alumno llevará a cabo la reserva de los 3 buffers necesarios, A, B y C. Donde **A** será la entrada, de tamaño **A=BATCH x IN_DIM**, **B** los pesos de tamaño **B=IN_DIM x OUT_DIM**, y **C** la salida de tamaño **C=BATCH x OUT_DIM**. El tipo de datos de los buffers será de tipo flotante de simple precisión (*Nota: cuidado con el desbordamiento de los tamaños del buffer*).

Una vez hecha la reserva de los buffers, inicializamos cada una de las posiciones con valores. En este caso tendríamos las lecturas de los sensores, los pesos de las capas, etc. Como es una simulación, inicializamos los valores de entrada y salida a 0.01f, y los valores de los pesos a 0.02f.

EJERCICIO 2

En el siguiente paso, se implementará la capa forward, que corresponde al producto matricial, junto con la suma del bias. Siguiendo la expresión matemática presentada, el alumno llevará a cabo su implementación. Para ello, debe tener presente que la matriz de entrada A se considera que está almacenada en formato *row-major*. Esto implica que las muestras de un mismo sensor o los píxeles de una misma imagen se almacenan de forma consecutiva, o lo que es lo mismo, formarán la fila de una matriz. Por consiguiente, el número de filas vendrá dado por el tamaño de batch.

Nota: No olvide sumar al producto de cada entrada el bias correspondiente:
C = (A x B) + Bias según la definición.

$$z = \left(\sum_{i=1}^n w_i \cdot x_i \right) + b$$

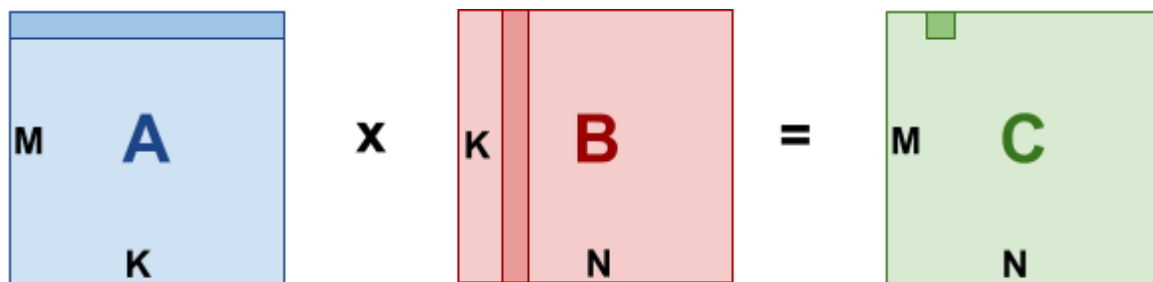


Figura 1. Esquema del producto matricial GEMM.

EJERCICIO 3

Una vez implementada la capa lineal, el alumno implementará la capa ReLU. Esta capa recibirá la salida de la capa anterior, es decir, la matriz C, y aplicará la función por definición:

$$f(z) = \max(0, z)$$

EJERCICIO 4

Como último ejercicio, se temporizará cada una de las dos capas que forman parte de nuestro MLP por separado y se llevará a cabo un estudio de cómo afecta el tamaño de batch al tiempo de procesamiento. Montar una gráfica donde el eje Y sea el tiempo de procesamiento y el eje X el tamaño de batch. La gráfica debe mostrar cómo crece el tiempo de la capa lineal en función del tamaño de batch.

Nota: Es importante que el alumno lleve a cabo los estudios de rendimiento en la misma plataforma durante el transcurso de todas las prácticas de la asignatura.

PRÁCTICA 3

La jerarquía de memoria

CONTEXTO

Hasta el momento, el alumnado ha aprendido la importancia de la serialización de los datos, así como a acceder a ellos de forma óptima y adecuada. Como se verá en las próximas prácticas, tanto para MPI como para CUDA, la serialización de los datos es de vital importancia a la hora de llevar a cabo las transferencias de estos. Esto se debe a que para optimizar los envíos siempre es recomendable realizar el menor número de envíos del mayor tamaño posible.

En esta práctica, el alumno observará el impacto del acceso a los datos almacenados en memoria. Como ya es conocido el alumno, los servidores se componen de diferentes niveles de memoria, tal y como muestra la Figura 1.

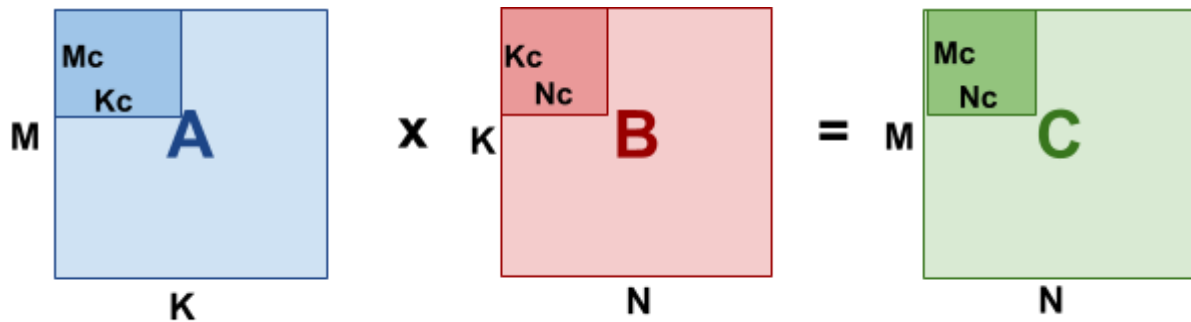


Fuente: https://es.wikipedia.org/wiki/Jerarqu%C3%ADa_de_memoria

Figura 1. Jerarquía de memoria de un computador.

Tal y como se muestra en la Figura 1, conforme los datos se encuentren más cercanos a la CPU, el acceso a ellos va a ser más rápido, aumentando la velocidad de transferencia y decreciendo la latencia de acceso.

Por este motivo, las bibliotecas más usadas de álgebra lineal para el procesamiento de redes neuronales u otro tipo de cálculos (BLIS, OpenBLAS, XNNPACK, etc.), usan un esquema especial para el producto matricial basado en bloques, el cual se extiende de CPU a GPU en bibliotecas como CUBLAS. Para esto, en vez de procesar el producto matricial completo, llevando a cabo el producto completo de $M \times N \times K$ elementos, se llevan a cabo productos de $M_c \times N_c \times K_c$, tal y como muestra la Figura 2. Además, en versiones más avanzadas, para llevar a cabo el producto con instrucciones SIMD, los bloques de la matriz $M_c \times K_c$ y $K_c \times N_c$ ordenan los datos (se empaquetan) de forma que el acceso sea directo y las cargas de los valores sean secuenciales. El desarrollador siempre ha de buscar un equilibrio, ya que si el tamaño de la matriz es reducido, no se llegaría a amortizar el coste del empaquetado de los datos.



```

1 void Gemm(int m, int n, int k, double *A, double *B, double *C) {
2     // Declarations: mc, nc, kc,...
3     for ( jc = 0; jc < n; jc += nc )           // Loop 1
4         for ( pc = 0; pc < k; pc += kc ) {      // Loop 2
5             // B(pc:pc+kc-1,jc:jc+nc-1) → Bc
6             Pack_buffer_B(kc, nc, &B(pc,jc), &Bc);
7             for ( ic = 0; ic < m; ic += mc ) {  // Loop 3
8                 // A(ic:ic+mc-1,pc:pc+kc-1) → Ac
9                 Pack_buffer_A(mc, kc, &A(ic,pc), &Ac);
10                // Macro-kernel:
11                for ( jr = 0; jr < nc; jr += nr ) // Loop 4
12                    for ( ir = 0; ir < mc; ir += mr ) { // Loop 5
13                        // Micro-kernel:
14                        // Cc(ir:ir+mr-1,jr:jr+nr-1) +=
15                        // Ac(ir:ir+mr-1,1:1+kc-1) ·
16                        // Bc(j,1:1+kc-1,jr:jr+nr-1)
17                        Gemm_mkernel( mr, nr, kc, &Ac(ir,1), &Bc(1,jr),
18                                    &Cc(ir,jr) );
19                    }
20                }
21            }
22 }

```

Figura 2. Producto matricial por bloques.

OBJETIVOS

Esta práctica se dividirá en dos partes principales. En primera parte, el alumnado implementará su propio benchmark para obtener el ancho de banda a la memoria principal del sistema (memoria RAM), así como a los diferentes niveles de memoria caché. En la segunda parte, el alumno llevará a cabo una nueva implementación del producto matricial, pero esta vez por bloques, siguiendo el esquema presentado en la Figura 2 de esta práctica.

Dicho esto, los objetivos principales de la práctica son:

- Conocimiento del alumnado de los diferentes niveles de memoria disponibles en un sistema.
- Impacto en el rendimiento en función de los accesos a memoria.
- Comprensión del producto matricial por bloques.

EJERCICIO 1

El objetivo de este primer ejercicio es que el alumno vea la relación entre los diferentes tipos de acceso a la memoria que se llevan a cabo en ciencia de datos y los tamaños de memoria caché del sistema. Para ello, el primer paso será obtener la información desglosada del tamaño de los diferentes niveles de memoria caché de su sistema.

Nota: Recuerde usar la misma plataforma que en la práctica anterior.

En sistemas operativos Linux, existen herramientas como **lscpu**, **lstopo (hwlock)**, etc., que nos pueden ayudar a obtener todo este tipo de información.

A continuación, rellene la siguiente tabla con la información solicitada.

Nivel de Memoria	Tamaño (MB)
L1 (datos)	
L2	
L3	
Memoria RAM	

Una vez obtenida esta información se procederá a la implementación del test para el ancho de banda. Para estandarizar las temporizaciones, se llevarán a cabo mediante la rutina:

```
static double now_sec(void) {  
    struct timespec ts;  
    clock_gettime(CLOCK_MONOTONIC, &ts);  
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;  
}
```

Mediante el uso de la misma, el alumno obtendrá el tiempo en segundos del fragmento de código deseado:

```
double t0 = now_sec();  
//Fragmento de código  
double t1 = now_sec();
```

Para llevar a cabo la implementación del test del ancho de banda y del impacto de los accesos, el alumno deberá, en primera instancia, reservar un tamaño de buffer suficientemente grande para sobrepasar el tamaño del último nivel de caché L3 (por ejemplo, 1 GB). A continuación, se implementarán diferentes rutinas, las cuales irán accediendo a los datos del vector y operando con ellos.

Para esto, la primera implementación llevará a cabo un acceso secuencial a los datos e irá acumulando el contenido de cada posición del vector en una variable local "sum".

```
FOR i HASTA n  
    sum += buf[i]
```

Para la segunda implementación, se creará un vector con números aleatorios (dentro del tamaño del vector que se vaya a procesar) y la rutina, en vez de acceder a valores consecutivos, accederá a la posición que indique el valor aleatorio.

```

FOR i HASTA n
    aleatorio[i] = rand() % n
...
FOR i HASTA n
    sum += buf[aleatorio[i]]

```

Nota: En el segundo caso únicamente se temporizará el acceso y acumulación (segundo bucle), no la inicialización.

Nuestro objetivo es ver el comportamiento del ancho de banda en función del tipo de acceso y del tamaño del buffer al que accedemos. Se ha indicado que la reserva es de 1 GB, pero esto únicamente es el espacio en memoria reservado. Por lo tanto, para este objetivo, se creará un bucle que irá marcando el crecimiento de 'n'. Para esto, empezaremos con tamaños de 4, 8, 16, 32, 64, 256 y 512 Kb, y seguidamente, 4, 8, 16, 32, 64, 256 y 512 Mb. Para cada uno de estos tamaños, se temporizará el coste de las dos rutinas de acceso por separado y se mostrará tanto el tiempo consumido como el ancho de banda. Para el cálculo del ancho de banda, el alumnado debe recordar que la fórmula responde a datos transferidos por unidad de tiempo, donde se reportarán los datos transferidos en Mb y el tiempo en segundos **Mb/s** (**Nota: ¡cuidado con las unidades!**).

Resumidamente, el alumnado deberá reportar una tabla de tiempos similar a esta para cada una de las dos implementaciones.

Datos Transferidos (Mb)	Tiempo (s)	Ancho de Banda (Mb/s)
0.004 (4KB)		
...		
512		

Recomendaciones:

- Los tiempos de acceso en muchos casos son muy pequeños, se recomienda para cada test llevar a cabo una serie de repeticiones y contabilizar todo en el tiempo. Al final, se dividirá el tiempo total entre el número de repeticiones llevadas a cabo.
- El compilador puede optimizar el código si detecta que la acumulación que se lleva a cabo internamente en los bucles no se utiliza posteriormente en el programa. Se recomienda crear una variable global:

```
static volatile uint64_t g_sink = 0;
```

Y asignar el resultado de la variable sum a dicha variable fuera de la temporización. En caso de crear rutinas independientes y devolver el valor resultante, no sería necesario.

- Se recomienda utilizar las opciones de compilación: **-O2 -std=c11**

EJERCICIO 2

Implemente el producto matriz-matriz o GEMM desarrollado en la práctica anterior siguiendo la estrategia por bloques presentada en la Figura 2 de esta práctica.

PRÁCTICA 4

Evaluación de sistemas informáticos

CONTEXTO

En este punto, el alumnado ha implementado su propio producto matricial necesario para llevar a cabo un MLP, además, ha generado su propio test para obtener el ancho de banda real de la memoria RAM. En este punto, es hora de evaluar el rendimiento de nuestra implementación en el sistema en el que se está ejecutando.

Como expertos en plataformas de big data, debemos procesar el mayor volumen de datos en el menor tiempo posible explotando todos los recursos del servidor. Para ello es imprescindible conocer los mecanismos necesarios para evaluar el rendimiento de las aplicaciones desarrolladas por nosotros mismos, o por empresas externas. Para esto, el alumno aplicará las técnicas aprendidas hasta el momento para evaluar el rendimiento de sus propias implementaciones.

OBJETIVOS

En este contexto, el alumno en esta práctica deberá llevar a cabo un estudio de rendimiento de su propia implementación del producto matricial calculando métricas como el tiempo de procesamiento, FLOPs, memoria consumida, intensidad aritmética, etc., y así, posteriormente, relacionar las medidas en un modelo roofline.

Los objetivos principales de esta práctica:

- Conocer cómo se lleva a cabo el cálculo de FLOPS de un algoritmo.
- Familiarizarse con las aplicaciones disponibles en S.O Linux para observar el rendimiento del sistemas y las aplicaciones.
- Saber reflejar las métricas obtenidas en una gráfica roofline para así estudiar la relación del rendimiento con la intensidad aritmética y el pico de rendimiento del sistema.

EJERCICIO 1

En primer lugar, a partir de la implementación del MLP, procederemos a evaluar el producto matricial. Para poder llevar a cabo una evaluación exhaustiva, en primer lugar se necesita que el tiempo de ejecución sea lo suficientemente grande como para conseguir una estabilidad y no obtener excesiva variabilidad entre ejecuciones. Para ello, el alumno puede incrementar el tamaño de batch hasta que el tiempo de ejecución del producto matricial sea el deseado.

Una vez obtenido un tiempo de ejecución deseado y estable entre ejecuciones, procederemos a calcular los FLOPS de la operación. Para ello, como el alumno ya es conocedor, los FLOPS son operaciones en coma flotante por segundo, en este caso, al tratarse de datos en coma flotante de simple precisión, podremos obtener este tipo de métrica.

Para llevar a cabo el cálculo, necesitamos saber qué operaciones realizamos:

$$z = \left(\sum_{i=1}^n w_i \cdot x_i \right) + b$$

Esto implica una multiplicación y una acumulación: **A x B + Bias**. Es decir, **2 * M * N * K (operaciones)**. Como ya conocemos, para obtener los FLOPS:

$$\text{FLOPS} = T \text{ (s)} / (\text{Operaciones en coma flotante})$$

Como la unidad será muy elevada, se llevará a cabo la transformación a GFLOPS:

$$\text{GFLOPS} = \text{FLOPS} / 10^9$$

Dicho esto, el alumno reportará junto al tiempo que tarda el procesamiento los GFLOPS del producto matricial.

EJERCICIO 2

Mientras se está ejecutando el producto matricial (se puede incrementar más el tamaño de batch para observar mejor el comportamiento), se abrirá en una segunda ventana un terminal con el comando **top/htop**.

Con esto se pretende observar el comportamiento de la aplicación, viendo el porcentaje de consumo de CPU, así como el consumo de memoria RAM, etc.

EJERCICIO 3

En este punto, el alumno ya tiene todos los ingredientes para poder combinarlos y montar una gráfica roofline. Para el ancho de banda se tomarán los valores calculados en la pasada práctica (en modo secuencial) y para los valores de rendimiento los ya obtenidos en el ejercicio anterior.

Recordad que es necesario inicialmente calcular la Intensidad Aritmética (IA), así como el rendimiento pico de la máquina.

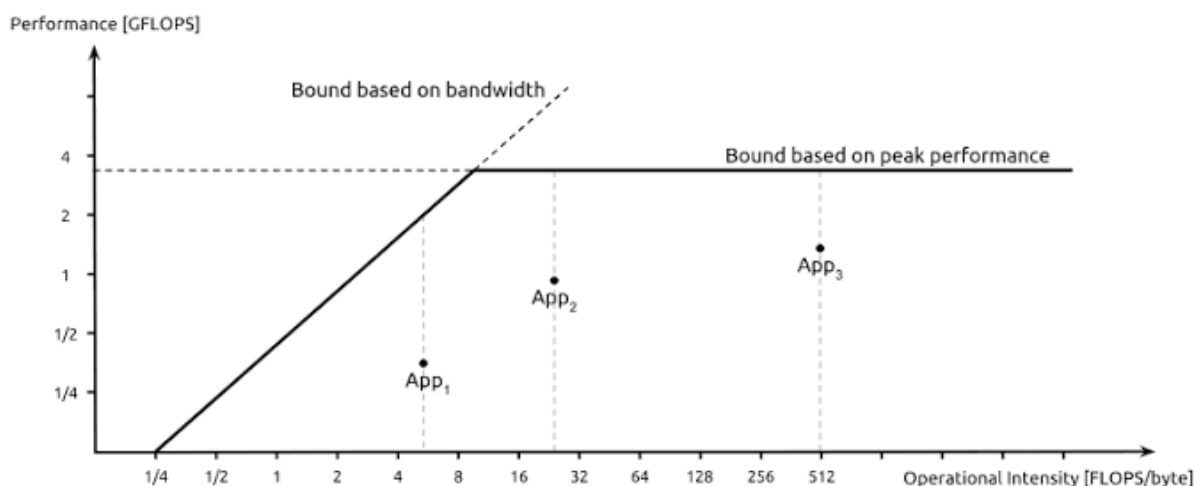


Figura 1. Gráfica representativa del Roofline, obtenida de:

https://en.wikipedia.org/wiki/Roofline_model

EJERCICIO 4

El presente ejercicio es un reto para el alumnado. En primer lugar, el alumno debe optimizar con sus conocimientos actuales el producto matricial desarrollado inicialmente. **QUEDA TOTALMENTE PROHIBIDO EL USO DE INTERNET Y CUALQUIER TIPO DE IA EN ESTA PARTE DEL EJERCICIO.**

Una vez el alumno esté conforme con su implementación, el alumno tomará el tiempo y el rendimiento en GFlops obtenidos para su implementación y ya podrá hacer uso de internet y cualquier IA para desarrollar una versión óptima del producto matricial. De igual manera, el alumno tomará los rendimientos para esta versión.

Finalmente, el alumno integrará una biblioteca dedicada de álgebra lineal como:

<https://github.com/flame/blis>

No se detallan los pasos para su instalación para que el alumnado se acostumbre a la lectura de documentación de bibliotecas externas y a la compilación de la misma.

Seguidamente, en vez de llevar a cabo la llamada a nuestra propia rutina del producto matricial, se llevará a cabo la llamada a la rutina GEMM de BLIS: **sgemm_**. Una vez llevada a cabo la implementación, se evaluará de nuevo. En la evaluación se recomienda, en primer lugar, fijar la variable `export OMP_NUM_THREADS=1`, antes de la ejecución, y una vez evaluado, fijar `export OMP_NUM_THREADS=n`, siendo `n` el número de procesadores físicos del sistema.

Cree una gráfica de barras, donde el eje `y` sea los GFLOPS obtenidos para cada implementación matricial probada (la propia del alumno, la propia de la IA generativa y la propia de BLIS). Además, se incluirán en la gráfica del roofline los nuevos puntos obtenidos para ambas implementaciones.

PRÁCTICA 5

Sistemas de memoria compartida

CONTEXTO

En la pasada práctica, el alumno evaluó diferentes implementaciones de una gemm y la relación de rendimiento frente al pico de la máquina.

La presente práctica se centra en la programación en sistemas de memoria compartida; esto significa que los hilos que intervengan en el procesamiento van a ver y compartir la misma memoria principal del sistema, y por lo tanto, los datos almacenados.

Para llevar a cabo el desarrollo de esta práctica, se utilizará la biblioteca OpenMP y habrá que tener siempre en cuenta incluir en la cabecera del fichero:

```
"#include <omp.h>"
```

Así como la opción -fopenmp en la compilación del binario.

OBJETIVOS

En este escenario, se le plantea al alumnado una serie de ejercicios para trabajar los conceptos aprendidos en clase referentes a la paralelización de los datos mediante el uso de hilos ligeros. Dicho esto, los principales objetivos de esta práctica son:

- Controlar las sentencias principales que ofrece OpenMP para la paralelización de secciones de código.
- El alumno debe aprender a aplicar las principales técnicas de sincronización de hilos.
- Aplicar técnicas de reducción de resultados fundamentales en plataformas de Big Data y procesamiento paralelo.

EJERCICIO 1

Este primer ejercicio consistirá en llevar a cabo una implementación paralela del producto matricial desarrollado por el alumnado en la red neuronal MLP (la versión original, sin bloques).

Una vez desarrollado, se llevará a cabo un estudio de escalabilidad. Para esto, se elegirá un tamaño de batch suficientemente grande para que todos los hilos tengan trabajo suficiente. A continuación, se observará el comportamiento del sistema con lanzamientos paralelos; para ello, se fijará el número de hilos al mismo número de núcleos del sistema y se llevarán a cabo los lanzamientos, contrastando el resultado con htop/top... Una vez analizado el comportamiento de la aplicación y el sistema, se estudiará la escalabilidad, temporizando y aumentando progresivamente el número de hilos involucrados en la ejecución.

Para observar la aceleración, se obtendrá la métrica del factor de aceleración denominada comunmente *speed-up*.

Speed-up = T (s) versión base / T(s) versión acelerada

Esto proporcionará el factor de aceleración respecto a la versión secuencial, en este caso. El speed-up ideal que se busca en la paralelización de un algoritmo es lineal respecto al número de núcleos involucrados en la ejecución. Esto implica que para 1 hilo la aceleración será 1, para 2 hilos 2, para 3 hilos 3, etc.

Finalmente, lleve a cabo el estudio de los resultados generando una gráfica de líneas con los resultados obtenidos.

EJERCICIO 2

La paralelización del producto matricial, sin empaquetado, como ha podido observar el alumnado es un proceso trivial con OpenMP, mediante una única directiva somos capaces de paralelizar el trabajo que se lleva a cabo dentro de varios bucles. Pero, ¿qué bucle decidió paralelizar? ¿Por qué? ¿Ha probado otro bucle diferente?

Lleve a cabo este estudio observando la diferencia de rendimiento entre las implementaciones realizadas.

EJERCICIO 3

Una vez contestadas las preguntas se plantea al alumno la paralelización del siguiente código secuencial buscando la máxima escalabilidad / rendimiento.

El alumno deberá implementar su propio programa, junto con las reservas de memoria pertinentes y la declaración de variables para que sea totalmente funcional.

Nota: Como una complicación extra, la paralelización deberá realizarse de manera manual, es decir, se crearán previamente los hilos con una sección. Queda **PROHIBIDO** el uso de la directiva *#pragma omp parallel for*.

```
#pragma omp parallel {}
```

Y posteriormente, mediante el identificador de los hilos se llevará a cabo el reparto de la faena. Esto implica que no se podrá hacer uso de las cláusulas como: `parallel for`, `reduction`, etc.

Código para su implementación paralela:

```
for (long i = 0; i < N; i++) { suma += array[i]; }
```


PRÁCTICA 6

Sistemas de memoria distribuida

CONTEXTO

Esta práctica se centra en sistemas de memoria distribuida. Para ello, se ha simulado un sistema clúster mediante una placa Jetson Nano dotada de una GPU que hace el papel de front-end y otras dos placas Raspberry Pi como nodos de cómputo del clúster. Ambas plataformas son ARM y están conectadas mediante una red privada a través del router, como puede verse en la Figura 1.



Figura 1. Montaje de un clúster formado por una conexión en red entre dos Raspberry Pi y una Jetson Nano.

Como puede observar el alumno en la Figura 1, el clúster está dotado de un router que establece una red privada entre los distintos elementos que lo conforman. Este router dispone, a su vez, de conexión Wi-Fi, que el alumnado utilizará para conectarse de forma remota al clúster señalado. Para ello, las credenciales de acceso son:

NOMBRE WIFI: PBDServer

CONTRASEÑA: qwerty1234

El clúster, como se ha comentado anteriormente, está formado por 2 RaspBerry Pi, los nodos de cómputo, y una Jetson Nano, nodo de acceso y front-end. Para comprender mejor la topología, se presenta su estructura en la Figura 2.

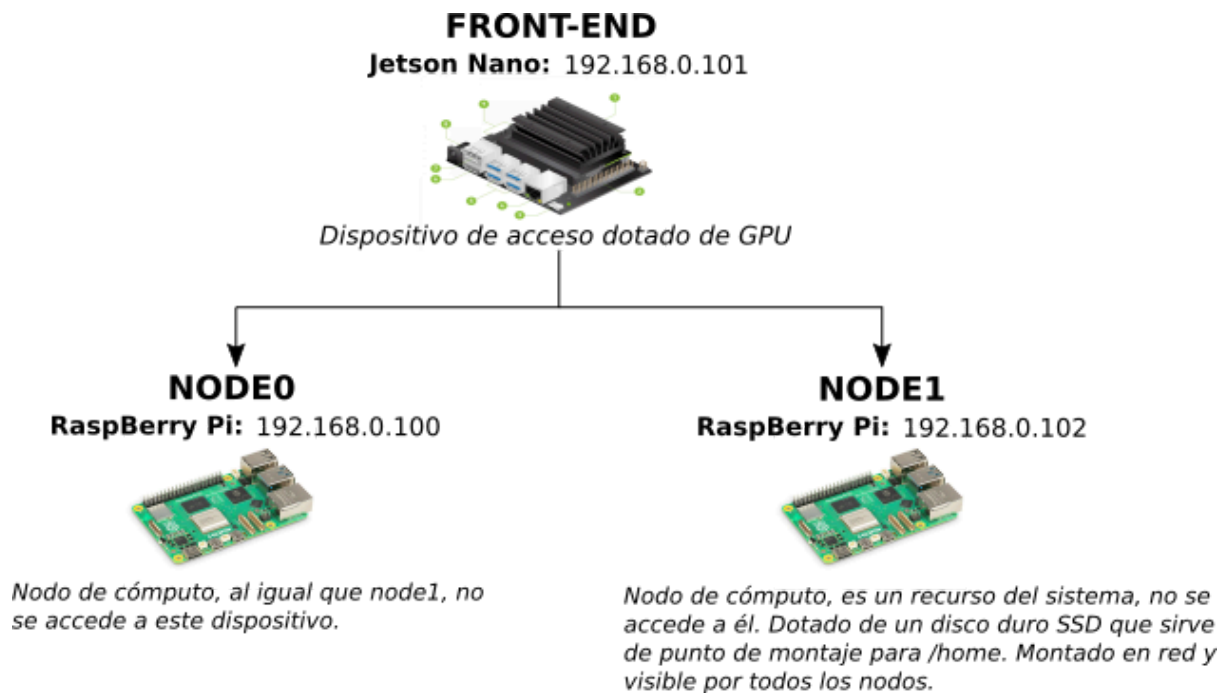


Figura 2. Esquema de interconexión de los componentes del clúster.

Como muestra la presente figura, la idea es representar a pequeña escala el esquema de un gran clúster (como podría ser el Mare Nostrum) y que de esta manera el alumnado se familiarice con este entorno de trabajo profesional. Para ello, se ha dotado al clúster de una Jetson Nano con GPU que hace el papel de front-end. El término front-end hace referencia al dispositivo o nodo del clúster a través del cual se lleva a cabo el acceso al mismo, y desde el cual se lleva a cabo el lanzamiento de trabajos (aunque no tiene por qué ser siempre el nodo a través del que se acceda al sistema). En nuestro escenario, el alumno, en primer lugar, debe

llevar a cabo la conexión de su PC a la red Wi-Fi del sistema para, posteriormente, acceder a través de la dirección IP 192.168.0.101 de esta red privada. Las credenciales de acceso serán:

USUARIO: alXX (el identificador otorgado por el profesor)

CONTRASEÑA: qwerty1234

La conexión, por lo tanto, se llevará a cabo mediante el protocolo SSH a través del puerto 22 a la ip 192.168.0.101 de la siguiente manera:

\$ ssh -p 22 192.168.0.101

Una vez dentro del front-end, el alumno verá en primer lugar el mensaje de bienvenida al clúster así como cierta información del sistema (nombre del host, kernel, CPU, memoria RAM, temperatura, número de usuarios conectados, IP, fecha, hora, etc.), tal y como muestra la Figura 3.

J E T S O N N A N O F R O N T - E N D	
HOST	: frontend
KERNEL	: 4.9.253-tegra
UPTIME	: up 17 minutes
CPU	: 3%
RAM	: 290M / 1,9G (15%)
TEMP	: 36°C
DISK	: 31%
USERS	: 1
IP	: 192.168.0.101
DATE	: 2021-12-10
TIME	: 10:32:52

Figura 3. Mensaje de bienvenida del Front-end.

Este mensaje de bienvenida es configurable por el administrador del sistema, en este caso, el profesor, por lo tanto, puede adaptarse en función de las necesidades de los usuarios del cluster (como si se desea incluir un ascii art, tal y como muestra la Figura 4).

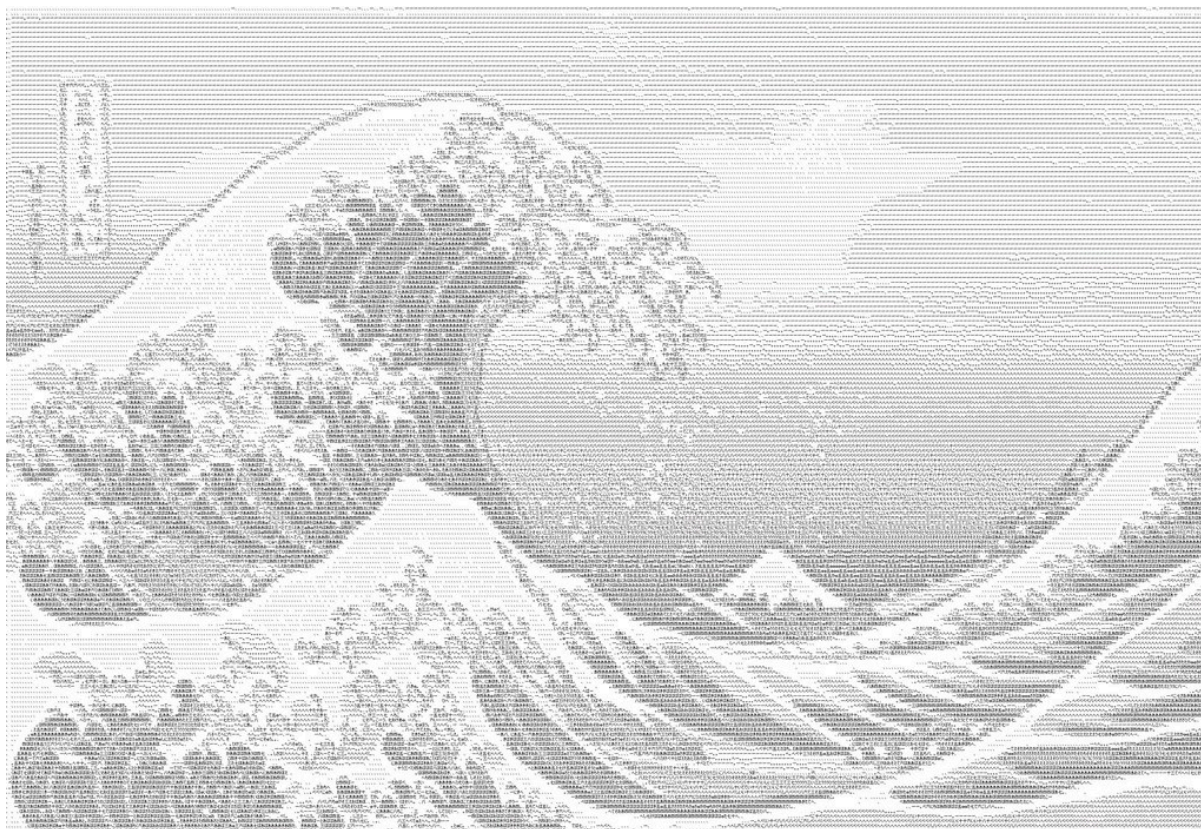


Figura 4. Ascii art para la pantalla de bienvenida.

Tanto este mensaje de bienvenida como las diferentes variables que se necesiten cargar para el correcto funcionamiento del sistema se pueden configurar en scripts personalizados dentro de la ruta **/etc/profile.d/mi_script.sh**.

El siguiente paso será observar las características del sistema. En primer lugar se observará cómo está montado el sistema de ficheros. Como se detalla en la Figura 2, el disco duro principal del clúster se encuentra localizado en el nodo 1. Esto no quita que cada elemento del clúster disponga de su propio disco duro (en este caso, la tarjeta SD donde está instalado el sistema operativo). Para que pueda funcionar correctamente esta configuración, el disco duro SSD está montado en red, por lo tanto, si se lleva a cabo el comando:

\$ df -h

El alumno podrá observar las diferentes características y puntos de montaje del propio sistema, tal y como se muestra en la Figura 5. En la salida de dicho comando puede observarse como el sistema de ficheros donde está montado **/home** procede del nodo **192.168.0.102**, el cual dispone del disco duro SSD.


```
martineh@frontend:~ $ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/mmcblk0p1  59G   17G   39G   31% /
none            951M    0    951M    0% /dev
tmpfs           990M   4,0K   990M    1% /dev/shm
tmpfs           990M   20M   971M    2% /run
tmpfs           5,0M   4,0K   5,0M    1% /run/lock
tmpfs           990M    0    990M    0% /sys/fs/cgroup
tmpfs           198M   4,0K   198M    1% /run/user/121
192.168.0.102:/home 916G   2,1G   868G    1% /home
tmpfs           198M    0    198M    0% /run/user/1000
```

Figura 5. Ejemplo de salida por pantalla del comando `df`. Muestra como está montado el sistema de ficheros del cluster de computadores.

Por otro lado, un punto importante es conocer el nombre de los nodos, para ello, dentro del fichero de configuración `/etc/hosts`, el alumno podrá observar los diferentes nodos, IPs y nombres de los dispositivos que conforman el clúster. Cabe remarcar que este fichero únicamente es modificable por el administrador del sistema, el cual poseerá las credenciales de root.

Los 3 dispositivos que conforman el clúster tienen arquitectura ARM, pero no todos ellos disponen de las mismas características. Por ejemplo, el front-end dispone de GPU pero no de los nodos de cómputo. Además, la Jetson Nano es un dispositivo mucho más antiguo que el resto de nodos, más concretamente, para tener una versión de cuda funcional, el dispositivo dispone de Ubuntu 18.04, mientras que la Raspberry Pi dispone de sistemas operativos más actuales como Ubuntu 22.04. Esto se convierte en un problema a la hora de la administración de aplicaciones, ya que tanto para la compilación de aplicaciones como para el gestor de colas, todos los nodos que componen un clúster deben disponer de versiones similares de software para evitar incompatibilidades.

Dicho esto, como se ha detallado, los tres nodos que componen el clúster disponen de arquitectura ARM, esto tiene una especial relevancia a la hora de compilar nuestros binarios. Cuando el alumno vaya a ejecutar sus aplicaciones MPI en distribuido, el binario debe estar disponible en todos los nodos del clúster. ¿Ve el alumnado la problemática? Para evitar esto, los clústeres suelen disponer de un espacio compartido visible para todos los nodos que comparten arquitectura de procesador (como en nuestro caso). De esta forma, el alumno llevará a cabo la compilación del binario sobre el front-end, y al ser un espacio compartido, el resto de nodos que llevarán a cabo la ejecución del binario tendrán acceso automáticamente

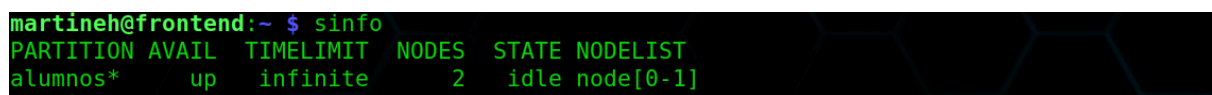
a este binario generado. Para llevar a cabo la compilación, el alumno tiene a su disposición el compilador de OpenMPI (mpicc) y el propio gcc.

Por último, y no por ello menos importante, nos centraremos en el sistema de colas, uno de los componentes más importantes del clúster. Una vez generado el binario, los alumnos harán uso del sistema de colas para lanzar sus trabajos. En este caso, el gestor de colas está basado en SLURM (uno de los más usados en la actualidad por su versatilidad y flexibilidad con los recursos del sistema), por lo tanto, será necesario conocer los conceptos básicos del mismo.

Es de vital importancia que el alumno sea consciente de la importancia virtual que tienen las ejecuciones de aplicaciones en clústeres de alto rendimiento. En muchas ocasiones, los usuarios miden rendimientos de cómputo, y por lo tanto, es imprescindible que se lleven a cabo ejecuciones aisladas. Por este motivo, está completamente **PROHIBIDO** saltarse el sistema de colas. Siempre que un usuario de un clúster quiera ejecutar su aplicación, lo hará mediante el script correspondiente de lanzamiento junto con la instrucción “sbatch” que se verá con detalle más adelante.

En primer lugar, si el alumno desea ver los recursos disponibles en el gestor de colas SLURM ejecutará el comando:

\$ sinfo



```
martineh@frontend:~ $ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
alumnos*   up       infinite    2      idle node[0-1]
```

Figura 6. Ejemplo de salida sinfo.

Como muestra la Figura 6, el alumno verá algo similar a esto: el gestor de colas nos muestra que tenemos dos nodos disponibles, nodo 0 y nodo 1, y ambos están en estado ocioso o desocupado. En este caso podríamos ver diferentes estados, entre los más habituales:

- **idle:** Nodo disponible.
- **alloc:** Nodo ocupado.
- **mix:** Parcialmente ocupado.
- **down:** Nodo no disponible o caído.

Podemos lanzar trabajos rápidamente mediante el comando srun, además de incluir aquí nuestras propias configuraciones (número de nodos, en qué nodos deseamos lanzar, número de cpus por nodo, tiempo máximo de reserva, etc.). Un ejemplo sería:

\$ srun --time=00:05:00 --cpus-per-task=2 python programa.py

En esta línea de ejecución se solicitarían 2 cpus para lanzar “programa.py” por un tiempo máximo de 5 minutos. Transcurrido este tiempo, los recursos se liberarían aunque la ejecución de nuestra aplicación no hubiera concluido, abortando así su procesamiento.

En nuestro caso, para un entorno más profesional, NO se llevará a cabo este tipo de ejecuciones, ya que dejaremos de guardar y tener registro de nuestras propias configuraciones. Para esto, el alumno creará su propio script de lanzamiento (son altamente configurable; en nuestro caso, usaremos un script base de ejemplo para los ejercicios propuestos).

Para ello el alumno preparará **lanza.sh** con las siguientes líneas:

```
#!/bin/bash

#SBATCH --job-name=estadisticas_mpi    # Nombre de la tarea
#SBATCH --partition=alumnos            # Particion del cluster
#SBATCH --nodes=2                      # Usar los 2 nodos (node0 y node1)
#SBATCH --ntasks=8                    # Número total de procesos MPI (4 por nodo)
#SBATCH --ntasks-per-node=4           # Procesos por nodo
#SBATCH --output=resultado_%j.log      # %j se sustituye por el ID del trabajo

#A continuación introduzca la línea de ejecución del binario

mpirun -np 2 --host node0,node1 ./ejecutable
```

Código 1. Código de ejemplo de script de lanzamiento de un trabajo en un sistema de colas gestionado por SLURM.

Esto sería un ejemplo básico de script de lanzamiento. El alumnado debe fijarse en cómo se ha elegido la partición de alumnos. En este caso, únicamente existe una partición del clúster, pero en entornos grandes de trabajo puede haber diferentes colas de trabajo con distintas máquinas de trabajo y el usuario debería seleccionar en cuál de ellas quiere encolar su trabajo.

Pese a que la ejecución se lleva a cabo de la forma clásica mediante el binario de OpenMPI “mpirun”, SLURM ofrece el comando `srun`, el cual facilitaría la línea de ejecución, ya que únicamente sería necesario fijar “**srun ./ejecutable**”. Cabe remarcar que este tipo de ejecución no está disponible en nuestro caso debido a

incompatibilidades entre versiones de software de los diferentes sistemas operativos.

Finalmente, una vez creado y configurado correctamente el script de ejecución, se llevaría a cabo su lanzamiento mediante el comando:

\$ sbatch lanza.sh

Una vez encolado el trabajo el usuario podrá consultar su estado mediante el comando:

\$ squeue

Obteniendo una salida similar a la mostrada en la Figura 7.

```
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        29      alumnos estadist    al29 PD      0:00      2 (Resources)
        30      alumnos estadist    al28 PD      0:00      2 (Priority)
        28      alumnos estadist    al30 R       0:05      2 node[0-1]
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        29      alumnos estadist    al29 PD      0:00      2 (Resources)
        30      alumnos estadist    al28 PD      0:00      2 (Priority)
        28      alumnos estadist    al30 R       0:07      2 node[0-1]
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        29      alumnos estadist    al29 PD      0:00      2 (Resources)
        30      alumnos estadist    al28 PD      0:00      2 (Priority)
        28      alumnos estadist    al30 R       0:13      2 node[0-1]
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        29      alumnos estadist    al29 PD      0:00      2 (Resources)
        30      alumnos estadist    al28 PD      0:00      2 (Priority)
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        30      alumnos estadist    al28 PD      0:00      2 (Resources)
        29      alumnos estadist    al29 R       0:01      2 node[0-1]
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        30      alumnos estadist    al28 PD      0:00      2 (Resources)
        29      alumnos estadist    al29 R       0:06      2 node[0-1]
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        30      alumnos estadist    al28 PD      0:00      2 (Resources)
        29      alumnos estadist    al29 R       0:15      2 node[0-1]
martineh@frontend:~ $ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        30      alumnos estadist    al28 R       0:03      2 node[0-1]
martineh@frontend:~ $
```

Figura 7. Ejemplo de salida del comando squeue.

Además, el usuario podrá ver el estado de su trabajo lanzado:

- **R (running):** Trabajo en ejecución.
- **PD (pending):** Trabajo en espera hasta que se liberen los recursos del sistema.
- **CG (completado):** Trabajo finalizado.

Si el usuario deseara cancelar su trabajo en el clúster, ejecutaría el comando:

\$ scancel id_trabajo

Finalmente, una vez finalizada la ejecución del propio trabajo, se creará un fichero de log en el directorio donde el usuario ha lanzado su aplicación con la salida de su ejecución.

IMPORTANTE: Los alumnos deben ser conscientes de las limitaciones del sistema presentado. Por este motivo, sean pacientes a la hora de obtener resultados; por ejemplo, la primera vez que se accede al fichero log de salida, es natural que tarde unos segundos en mostrar la información.

OBJETIVOS

Los principales objetivos de esta práctica se centran en:

- El alumno se familiarizará con la ejecución de aplicaciones paralelas en un entorno profesional de clusters de computadores.
- Se adquirirán los conocimientos básicos para el lanzamiento de trabajo sobre el gestor de colas y recursos SLURM.
- Ser capaz de desarrollar aplicaciones básicas, paralelas, mediante la biblioteca de programación distribuida MPI.

EJERCICIO 1

El primer ejercicio consistirá en llevar a cabo un programa simple que muestre como se ejecutan los diferentes procesos MPI en cada uno de los nodos. Para ello, crearemos un primer programa C con MPI “**ejemplo_mpi.c**”. Dentro de nuestro fichero escribiremos el Código 2. Finalmente, para comprobar si funciona nuestro ejemplo, en primera instancia se deberá compilar el propio código C, para ello se utilizará el comando:

mpicc ejemplo_mpi.c -o ejemplo_mpi

Finalmente, llevaremos a cabo la ejecución del mismo:

mpirun -np 2 --host node0,node1 ./ejemplo_mpi

```

#include <mpi.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char** argv) {
    //Inicialización del entorno MPI.
    MPI_Init(&argc, &argv);

    //Obtención el número total de procesos distribuidos.
    int size; MPI_Comm_size(MPI_COMM_WORLD, &size);

    //Obtención del identificador del proceso específico.
    int rank; MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    //Obtención del nombre físico de la máquina.
    char hostname[256]; gethostname(hostname, sizeof(hostname));

    //Mensaje desde el nodo.
    printf("Proceso %d de un total de %d ejecutándose en: %s\n",
           rank, size, hostname);

    //Finalizar el entorno MPI
    MPI_Finalize();
    return 0;
}

```

Código 2. Ejemplo de programa MPI.

EJERCICIO 2

En este segundo ejercicio el alumno deberá de crear un programa distribuido capaz de calcular la media y la desviación típica de los elementos de un vector. Para ello, se repartirá el trabajo entre los nodos del cluster (en este caso 2 nodos). El tamaño del vector será una variable parametrizable y fijada por línea de comando, su valor por defecto será de 2 millones de elementos (hasta que no funcione adecuadamente la aplicación se llevarán a cabo pruebas pequeñas para no congestionar las colas y el sistema de trabajo innecesario. El alumnado debe recordar que se encuentra en un sistema de trabajo compartido). Una vez se tenga desarrollado el algoritmo, se ejecutará haciendo uso del ejemplo de script de lanzamiento de Código 1.

Media

$$\bar{x} = \frac{\sum_{i=1}^N x_i}{N}$$

Desviación típica

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N}}$$

Nota: El alumnado debe recordar para que para hacer uso de ciertas funciones matemáticas desde C es necesario enlazar con la biblioteca matemática **-lm**.

EJERCICIO 3

Para este ejercicio se va a simular una red de sensores IoT desplegados en el cluster PBD. Para ello, se va a considerar que un nodo genera una lectura de un sensor determinado, este la enviará al siguiente nodo dicho valor para que lo procese y mande de vuelta el resultado obtenido. Esto se se va a ejecutar en forma de anillo lógico, patrón muy habitual en sistemas streaming (kafka, storm, Flink...).

Especificación

1. Se lanzarán 2 procesos MPI (uno en cada Raspberry Pi)
2. El programa simulara N rondas o iteraciones (por defecto 5).
 - a. El proceso 0 generará un valor aleatorio entero simulando una lectura de un sensor.
 - b. Este proceso enviará el valor generado al proceso 1 con la directiva MPI_Send.
 - c. El proceso 1 lo recibe con MPI_Recv, el sumará su propio rank, y lo devolverá al proceso 0 de nuevo con MPI_Send.
 - d. El proceso 0 recibirá el resultado con MPI_Recv e imprimirá el valor final de dicha ronda.
3. El alumno debe garantizar la sincronización de los procesos, es decir, un proceso no debe comenzar una nueva ronda sin haberse asegurado que ambos han finalizado la ronda anterior.
4. Cada proceso imprimirá por pantalla el nombre del hostname donde se está ejecutando, su rank y que valor ha enviado en cada ronda:

[Ronda %d] Proceso %d (%s) recibe %d y envía %d.

PRÁCTICA 7

Programación GPU (CUDA)

CONTEXTO

En esta práctica el alumno se centrará en el desarrollo sobre dispositivos GPU mediante la biblioteca CUDA. La práctica podrá desarrollarse tanto el cluster disponible para la asignatura como en los computadores personales del alumnado, siempre y cuando dispongan de algún dispositivo GPU de Nvidia.

Para los alumnos que vayan a llevar a cabo la práctica en el clúster PBDServer, se accederá al mismo desde el nodo front-end y en este caso se llevarán a cabo las ejecuciones en el propio dispositivo. El software necesario para llevar a cabo el desarrollo de la práctica está todo preparado, por lo tanto, no será necesaria la preparación de ninguna biblioteca o binario para la compilación de los binarios, así como su ejecución. Por otro lado, cabe remarcar y concienciar al alumno de que se encuentra en el propio front-end del clúster, y la preparación de sus algoritmos, como compilación y ejecución, se va a llevar a cabo en este mismo dispositivo (escenario que no suele ocurrir en un entorno profesional debido a que el front-end no suele ser un nodo de cómputo sino de gestión). Además, se trata de un dispositivo jetson-nano, lo que puede suponer una gran carga de trabajo, por este motivo, el alumno debe de ser consciente y se paciente a que se completen sus ejecuciones, así como los tiempos de respuesta de la propia máquina. Por último, reseñar que se trata de CUDA 10, es decir, una versión bastante anticuada de CUDA, esto es debido a que Jetson-nano paró la actualización de sus propias placas en Ubuntu 18 y CUDA 10, por lo tanto no es posible llevar a una versión más actual de CUDA para su correcto funcionamiento. No obstante, esto no conlleva ningún tipo de impedimento para elaborar la práctica correctamente, únicamente que no estarán disponibles algunas de las funciones más actuales de CUDA, pero tampoco son necesarias para la elaboración de la presente práctica.

Para elaborar los ejercicios, el alumno en primer lugar deberá desarrollar sus propias implementaciones y así compilar mediante el compilador **nvcc** preparado en el sistema o en su propio computador.

Una vez generado el binario, este se lanzará en el propio dispositivo front-end, tal y como ya se ha descrito anteriormente.

OBJETIVOS

En esta práctica, el alumno desarrollará una serie de ejercicios para fortalecer y poner en práctica los conceptos adquiridos hasta el momento. Por esto, los principales objetivos consistirán en:

- El alumno se familiarizará con el entorno de desarrollo y ejecución en dispositivos GPU, por ello, desarrollará, compilará y ejecutará programas CUDA.
- El alumno adquirirá los conocimientos prácticos necesarios para la paralelización de aplicaciones sobre GPU, así como sus ventajas e inconvenientes.

EJERCICIO 1

Este primer ejercicio consistirá en comprobar el correcto funcionamiento de la GPU y la instalación de la propia biblioteca. Para ello, se proporciona al alumno un código de ejemplo CUDA disponible en el Código 1. Este código se deberá almacenar en un fichero con la extensión **.cu**, por ejemplo, **"hello_cuda.cu"**. Seguidamente, se compilará mediante la línea de comando:

\$ nvcc hello_cuda.cu -o hello_cuda

Una vez compilado sin errores, el ejemplo CUDA se ejecutará:

\$/hello_cuda

```

#include <stdio.h>
#include <cuda_runtime.h>

__global__ void hello_cuda() {
    printf("Hola desde GPU: bloque %d, hilo %d\n",
           blockIdx.x, threadIdx.x);
}

int main() {
    int deviceCount = 0;
    cudaError_t err = cudaGetDeviceCount(&deviceCount);
    if (err != cudaSuccess) {
        printf("Error CUDA: %s\n", cudaGetErrorString(err));
        return 1;
    }
    printf("GPUs CUDA detectadas: %d\n", deviceCount);
    if (deviceCount == 0) return 1;
    cudaDeviceProp prop;
    cudaGetDeviceProperties(&prop, 0);
    printf("GPU: %s\n", prop.name);
    printf("Compute Capability: %d.%d\n", prop.major, prop.minor);

    hello_cuda<<<1, 4>>>(); //Lanzamiento del kernel.

    err = cudaDeviceSynchronize();
    if (err != cudaSuccess) {
        printf("Error ejecutando kernel: %s\n",
               cudaGetErrorString(err));
        return 1;
    }
    printf("CUDA funciona correctamente.\n");
    return 0;
}

```

Código 1. Código de ejemplo CUDA.

EJERCICIO 2

Como ya se ha remarcado, la practica se desarrolla en un dispositivo Jetson Nano dotado con una GPU pequeña de 128 núcleos CUDA, por este motivo se trabajará con tamaños de vectores moderados. Para este primer ejercicio se llevará a cabo inicialmente la reserva en memoria RAM de 3 vectores (dos para las entradas y uno para la salida). Los vectores se inicializarán para posteriormente ser copiados a la memoria de la GPU. El tamaño de los vectores será parametrizable y su valor por defecto será de 128. Seguidamente, se desarrollará un Kernel GPU que lleve a cabo la suma y acumulación de dos de los vectores (A y B) sobre el tercer vector de salida (C). Finalmente, el alumno llevará a cabo la copia del vector resultado a la memoria RAM del Host para cerrar la aplicación con la liberación de la memoria.

Mostrar por pantalla, los resultados generados por el kernel.

EJERCICIO 3

Para este tercer ejercicio, se llevará a cabo un caso de uso típico de *edge computing*, el procesamiento de imágenes capturadas por una cámara antes de llevar a cabo un modelo de inferencia. Para ello tendremos una imagen 2D, la cual se generará sintéticamente al inicio del programa. Posteriormente se llevará a cabo el procesamiento en CPU para poder comparar posteriormente la salida de la GPU. Para aplicar el filtro se fijará un umbral, el cual será un valor parametrizable, y se aplicará siguiendo el algoritmo estándar del filtrado:

```
if (pixel >= umbral) 255  
  
else 0
```

Para el resto del ejercicio se procederá de forma similar al anterior en lo referente a movimientos de datos: Reserva memoria HOST + Creación de datos HOST + Movimiento de datos a HOST a GPU + Procesamiento GPU + Copia de datos de GPU a HOST